

MPLADS Sentinel

Smart India Hackathon — SIH26102

Explainable AI-Powered Audit Prioritization for MPLADS Projects

1. Problem Statement

Problem Statement ID

SIH26102

Problem

Monitoring a large number of MPLADS projects requires reviewing information across expenditure, implementation timelines, project locations, implementing agencies, and reported progress.

When thousands of projects are involved, manually reviewing every project with the same level of scrutiny is inefficient.

The key challenge is:

How can available project data be analyzed to identify projects that deserve audit attention first?

Our Approach

MPLADS Sentinel is an explainable audit-prioritization platform that analyzes MPLADS project data, identifies unusual patterns, ranks projects according to an Audit Priority Score, and provides evidence explaining why a project was flagged.

The system is designed to support human auditors rather than replace them.

Analyze → Detect → Prioritize → Explain → Investigate

2. Project Overview

MPLADS Sentinel processes curated MPLADS/eSAKSHI-derived project data and clearly labelled synthetic demonstration data through a data-processing and analytics pipeline.

The system evaluates projects across five analytical signals:

1. Cost anomaly
2. Timeline anomaly
3. Duplicate/spatial overlap

- 4. Agency risk
- 5. Progress/expenditure mismatch

These signals are combined into a transparent, rule-based **Audit Priority Score from 0 to 100**.

High-priority projects are presented to auditors through a priority queue, with supporting evidence available through a project investigation interface.

Core Principle

Sentinel identifies patterns that deserve attention; it does not declare fraud. Final decisions remain with human auditors.

3. Key Features

Audit Priority Scoring

Ranks projects using five independent analytical signals and produces a score between 0 and 100.

Explainable Anomaly Detection

Every flagged signal stores the underlying evidence used to calculate the score, allowing auditors to understand why a project received a particular priority.

Priority Queue

Automatically ranks projects so auditors can focus their attention on the highest-priority cases first.

Project Intelligence

Provides project-level information covering financials, implementation timelines, agencies, locations, anomalies, and audit priority.

GIS-Based Analysis

Visualizes project locations and supports spatial investigation of nearby or potentially overlapping works.

Investigation Workspace

Allows auditors to review evidence, add investigation notes, record findings, and close or escalate cases.

Data Ingestion

Supports CSV-based ingestion of curated project datasets through the FastAPI backend.

Audit Logging

Records important system actions such as dataset uploads, project access, investigation activity, and other relevant operations.

Synthetic Validation

Controlled synthetic anomalies can be introduced to evaluate whether the detection pipeline correctly identifies known abnormal patterns.

4. Audit Priority Signals

Signal	Maximum Points	Method
Cost Anomaly	30	Isolation Forest / percentile comparison
Timeline Anomaly	25	Isolation Forest / peer benchmark
Duplicate / Spatial Overlap	20	Geographic distance + text similarity
Agency Risk	15	Historical SQL aggregation
Progress / Expenditure Mismatch	10	Ratio-based rule
Total	100	

The initial weights are expert-defined MVP weights and are not claimed to be universally optimal.

Cost Anomaly

Identifies projects whose cost is unusually high compared with relevant peer projects.

Peer groups may consider:

- Work type
- District
- Project scale
- Comparable project characteristics

Timeline Anomaly

Identifies projects whose implementation duration deviates significantly from comparable projects.

Duplicate / Spatial Overlap

Identifies potentially overlapping or near-duplicate works using:

- Geographic proximity
- Project-description similarity
- Nearby comparable projects

Agency Risk

Aggregates historical anomaly patterns associated with implementing agencies.

This is a risk signal for prioritization and does not imply misconduct.

Progress / Expenditure Mismatch

Compares financial utilization against reported project progress.

Example:

```
Funds disbursed: 62%  
Work certified: 30%
```

A significant discrepancy can increase the project's audit priority.

5. Explainability

A central feature of MPLADS Sentinel is the ability to answer:

Why was this project flagged?

The system does not rely on an unexplained numerical prediction.

For every relevant signal, the system stores supporting evidence such as:

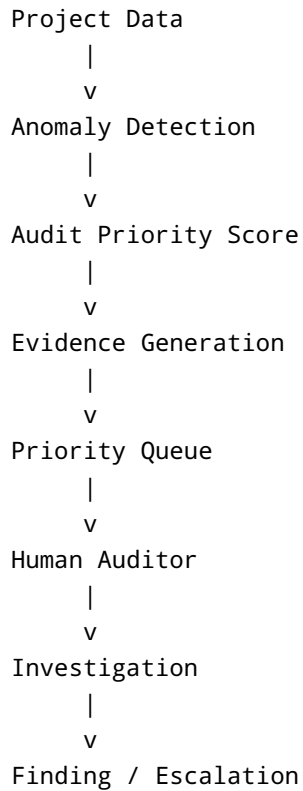
```
Cost deviation:  
34% above comparable-project median  
  
Timeline deviation:  
1.8× the peer-group benchmark  
  
Spatial signal:  
Similar project detected within the defined proximity threshold
```

The evidence is stored with the anomaly and score so that the frontend can display the explanation without recomputing the underlying analysis.

6. Human-in-the-Loop Investigation

MPLADS Sentinel is an audit-prioritization system, not an automated fraud-detection authority.

The intended workflow is:

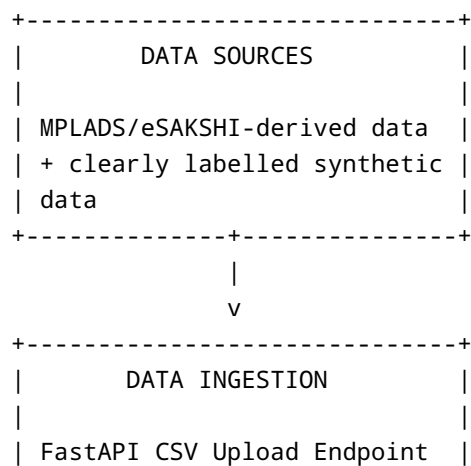


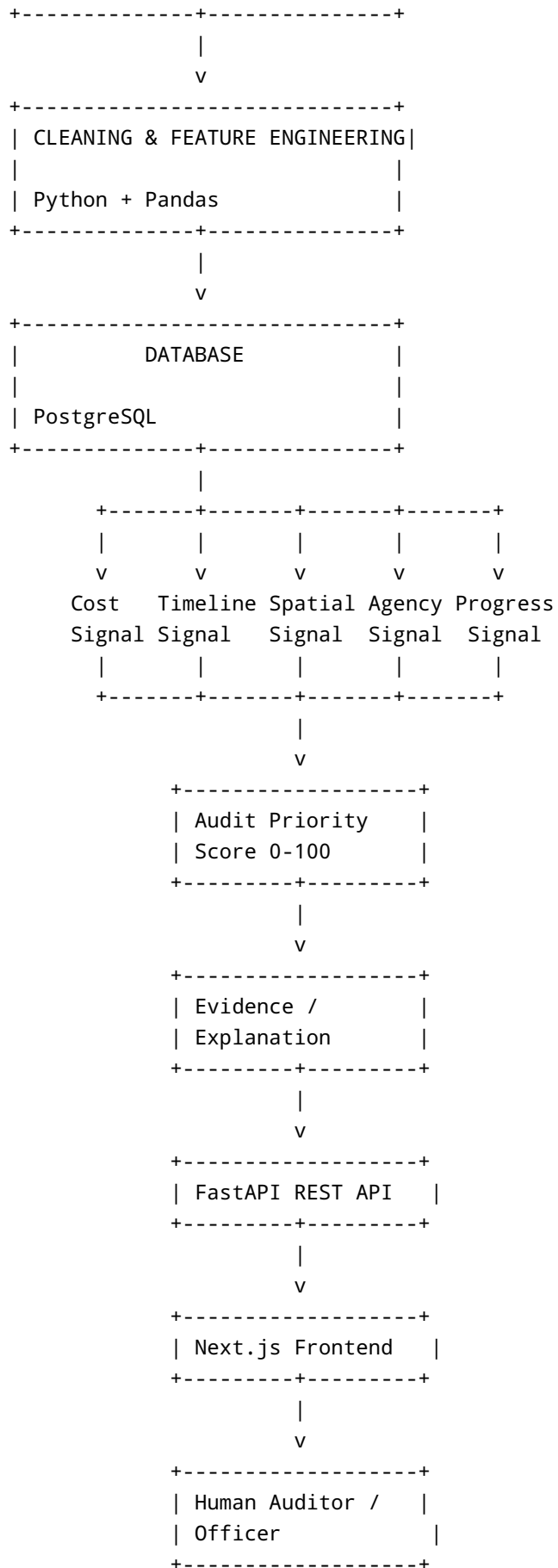
Possible investigation outcomes include:

- Under Review
- No Issue Found
- Requires Additional Information
- Verified Anomaly
- Escalated

A verified anomaly is not automatically equivalent to fraud.

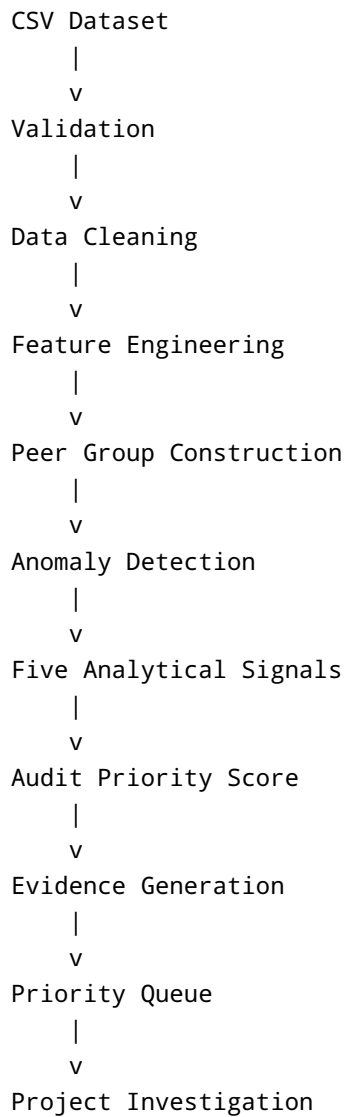
7. System Architecture





8. End-to-End Workflow

A typical project passes through the following pipeline:



9. Technology Stack

Layer	Technology
Frontend	Next.js / React
Frontend Language	TypeScript / TSX
Styling	Tailwind CSS
Charts	Recharts

Layer	Technology
Maps	Leaflet / React Leaflet
Icons	lucide-react
Frontend State	React State
Backend	Python / FastAPI
Data Processing	Pandas
Machine Learning	scikit-learn
Model Persistence	joblib
Database	PostgreSQL
ORM	SQLAlchemy
Validation	Pydantic
API	REST / JSON
Containerization	Docker Compose

10. Repository Structure

```

mplads-sentinel/
|
├── frontend/
|   ├── app/
|   ├── components/
|   ├── lib/
|   ├── styles/
|   ├── public/
|   ├── package.json
|   └── tsconfig.json
|
├── backend/
|   ├── app/
|   │   ├── main.py
|   │   ├── api/
|   │   ├── models/
|   │   ├── schemas/
|   │   ├── ml/
|   │   ├── data_pipeline/
|   │   └── database.py
|   ├── model_artifacts/
|   ├── data/
|   └── requirements.txt

```



```
|
| | docs/
| | | MASTER_ARCHITECTURE.md
| |
| | docker-compose.yml
| | .env.example
| | .gitignore
| | README.md
```

11. Data Sources

The MVP is designed around:

Public / Government-Derived Data

Curated MPLADS/eSAKSHI-derived datasets may be used for analysis and demonstration.

The source and nature of the dataset should always be clearly identified.

Synthetic Data

Synthetic datasets may be used to demonstrate controlled anomalies and validate the detection pipeline.

Synthetic records must always be clearly labelled.

The system must never present synthetic data as actual government records.

12. Validation Strategy

The absence of reliable fraud ground truth means that anomaly detection should not be presented as proof of fraud.

Instead, Sentinel can use controlled synthetic experiments.

Example:

Original project cost:	₹20,00,000
Injected synthetic cost:	₹45,00,000

The system can then evaluate whether the modified project receives a higher Audit Priority Score.

Such experiments should be explicitly labelled:

Synthetic Validation Experiment

Performance metrics can be reported for the synthetic experiment without implying equivalent performance on real-world fraud detection.

13. MVP Infrastructure Decisions

The project intentionally avoids unnecessary infrastructure for the SIH MVP.

PostgreSQL

PostgreSQL is used as the primary database.

PostGIS is not required for the initial implementation. Latitude/longitude fields combined with Python-based distance calculations are sufficient for the MVP spatial checks.

Local File Storage

Uploaded CSV files can be stored locally during the MVP.

S3/MinIO is not required.

Synchronous Processing

FastAPI can perform processing synchronously for the hackathon-scale MVP.

Celery/Redis is not required unless processing requirements later justify it.

Text Similarity

If sentence-transformers is too resource-intensive for the available development time, TF-IDF or difflib can be used as a lightweight MVP alternative.

14. Relationship Graph

A NetworkX-based relationship graph may be introduced as a future investigation capability.

Potential relationships include:

```
MP
|
+----- Project
|
|       +----- Agency
|       |
|       +----- District
```

The relationship graph is an investigation aid only.

It is **not an input to the MVP Audit Priority Score**.

15. MVP Scope

Included

- CSV data ingestion
- Data cleaning
- Feature engineering
- PostgreSQL database
- Cost anomaly detection
- Timeline anomaly detection
- Duplicate/spatial analysis
- Agency risk aggregation
- Progress/expenditure mismatch
- Audit Priority Score
- Evidence generation
- Dashboard
- Priority queue
- Project details
- GIS map
- Investigation workflow
- Audit logs

Not Included in MVP

- Live eSAKSHI integration
 - Automated fraud declarations
 - PostGIS dependency
 - S3/MinIO infrastructure
 - Celery/Redis workers
 - NetworkX scoring integration
 - Large-scale distributed processing
 - Unnecessary microservices
-

16. Installation and Setup

Prerequisites

Install the following:

- Git
- Node.js
- npm
- Python 3.9+
- PostgreSQL or Docker Desktop

Verify the installations:

```
git --version
node --version
npm --version
python --version
docker --version
```

Clone the Repository

```
git clone <repository-url>
cd mplads-sentinel
```

Backend

```
cd backend

python -m venv .venv
```

Activate the virtual environment.

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Install dependencies:

```
pip install -r requirements.txt
```

Frontend

```
cd ../frontend
npm install
```

Environment Variables

Create a local `.env` file using `.env.example` as the template.

Never commit real secrets to Git.

Database

Start the PostgreSQL service using the project's Docker Compose configuration:

```
docker compose up -d
```

Run the Backend

```
cd backend  
uvicorn app.main:app --reload
```

Run the Frontend

In a separate terminal:

```
cd frontend  
npm run dev
```

The development URLs will be displayed by the respective services.

17. Development Principles

The project follows these principles:

Explainability First

Every audit-priority signal should have an understandable reason and supporting evidence.

Human Decision-Making

The system assists auditors; it does not make final investigative or legal decisions.

Data Honesty

Synthetic and public datasets must never be misrepresented.

Reproducibility

Analytics and scoring should produce traceable and reproducible results.

Modular Architecture

Data processing, ML engines, backend APIs, and frontend components should remain modular.

MVP Discipline

Infrastructure should remain proportional to the 36-hour SIH development environment.

18. Demonstration Flow

The intended SIH demonstration is:

1. Open MPLADS Sentinel
2. View the monitoring dashboard
3. Open the Priority Queue
4. Select a high-priority project
5. View the Audit Priority Score
6. Select "Why was this flagged?"
7. Review the supporting evidence
8. Open the project on the GIS map
9. Start an investigation
10. Record the investigation outcome

The key demonstration message is:

MPLADS Sentinel does not make the final decision. It gives auditors an evidence-backed starting point for investigation.

19. Team

Member	Role
[Team Member 1]	[Role]
[Team Member 2]	[Role]
[Team Member 3]	[Role]
[Team Member 4]	[Role]
[Team Member 5]	[Role]
[Team Member 6]	[Role]

20. Project Status

Current Stage: Initial Repository Setup

The repository currently contains the project documentation and architecture reference.

Implementation will proceed incrementally across:

1. Repository setup
2. Backend foundation
3. Database schema
4. Data pipeline

5. Anomaly detection engines
 6. Audit Priority Score
 7. Evidence system
 8. REST API
 9. Frontend
 10. GIS integration
 11. Investigation workflow
 12. Testing and validation
 13. SIH demonstration preparation
-

21. Documentation

Detailed system architecture:

`docs/MASTER_ARCHITECTURE.md`

Additional technical documentation will be added as the implementation progresses.

22. License

This project is developed as part of the Smart India Hackathon.

License details will be added according to the team's chosen repository and project requirements.